



There's nothing to fix.

Software Engineer - Build It, Patch It, Ship It

Mission Brief

At Echo, we don't trust upstream to patch our images - we rebuild them from source so we can fix CVEs ourselves, on our own timeline. This assignment tests that you can do the full loop: compile a package from upstream, fix a vulnerability, ship a container image, and prove it's a drop-in replacement for the original.

Your mission is to ship a drop-in replacement for `nginx:1.25-bookworm` with **at least 2 CVEs eliminated**, where:

- **At least one** is fixed by **bumping the version** of a dependency (system library or upstream source)
- **At least one** is fixed by **backporting a patch** from a newer upstream commit onto the version you're shipping.

You'll also write an automated compatibility test that proves your image behaves like the original for a representative set of HTTP scenarios.

Objective

Produce a container image, the build artifacts that produce it, and a compatibility test that together prove you can:

1. **Build from source on Debian:** compiled output, not repackaged binaries from `apt`.
2. **Fix a CVE without an upstream release:** port a fix back to the version you're shipping.
3. **Prove the result still works:** automated tests, not manual inspection.

Please use at least two different remediation techniques (version bump and backport patch). Removing components is fine for extras, but it doesn't count toward the requirement.

Pull and Scan the Baseline

Pull the upstream image and scan it with both Trivy and Grype.
Save both reports; you'll diff against them later.

```
docker pull nginx:1.25-bookworm
trivy image nginx:1.25-bookworm > baseline-trivy.txt
grype nginx:1.25-bookworm > baseline-grype.txt
```

Triage

For each CVE in the reports, identify:

- The binary or library it lives in
- Whether upstream has a fix and in which version
- Whether you'd fix it by **version bump**, **backport patch**, or **removing the component**

Pick the CVEs you're going to fix and write down why.

Build the Package from Source

Write your own build script. You can lean on Debian tooling like `apt source`, `dpkg-buildpackage`, or `quilt` if helpful, but we'd rather see your own script:

- Fetch upstream source for the version you're targeting
- Apply your patches: version bump(s) + backport(s)
- Build the `.deb`

The build must run from a clean `debian:bookworm-slim` base, with no pre-baked binaries.

Wrap it in a `Makefile` (or single script) so we can reproduce it with one command.

Build the Container Image

Install your `.deb` into a minimal Debian base image.

Match the upstream image's filesystem layout, user, working directory, ports, and entrypoint **exactly**.

Your image needs to be a drop-in replacement, not a re-imagining.

Rescan and Compare (bonus)

Run Trivy and Grype on your fixed image and diff against the baseline.

Heads-up: scanners won't shrink your CVE list for backported fixes.

Trivy and Grype match by package name + upstream version, so an `nginx 1.25.x` package will keep getting flagged for the original CVE even after you've patched the source - the version string didn't change.

The right way to tell scanners "this CVE no longer applies to my build" is **VEX (Vulnerability Exploitability eXchange)**: a small attestation document next to the image declaring a CVE as `not_affected` / `fixed` with a justification. VEX-aware scanners (`trivy --vex`, `grype --vex`) honor it and drop the CVE from the report.

Emit a VEX document for one of your backport-patched CVEs and re-run the scanner with the VEX file applied to show the CVE actually disappear.

✓ Compatibility Test

Write an **automated** test (Go or Python) that:

1. Boots `nginx:1.25-bookworm` and your image as separate containers
2. Issues the same set of HTTP requests against each (root, custom config, large body, malformed request, etc.)
3. Asserts response status + headers + body match
4. Exits non-zero on any mismatch

Document what "working correctly" means and what your test covers.

📦 Deliverables

A single git repo containing:

1. `build/`: Dockerfile/script that produces the `.deb` from upstream source, plus a `patches/` directory with each backport patch named after the CVE it fixes.
 2. `Containerfile`: produces the final image from the `.deb`.
 3. `test/`: compatibility test code + a `make test` (or equivalent) target.
 4. `README.md` containing:
 - Build instructions
 - Image size: original vs. new
 - Per-CVE table: ID, severity, fix method (bump / backport / remove), evidence link
 - **Residual risk assessment**: what's still there, why, and what you'd do next
 - Anything that surprised you, or that you'd do differently with more time
-

💡 Reminders

- Focus on doing **a couple of CVEs end-to-end**, not many CVEs partially. Two well-fixed beats ten half-fixed.
 - This isn't production-grade. We're looking at the engineering, not the polish.
 - AI tools (Cursor, Claude Code, Copilot, ...) are encouraged! Tell us what you used them for and where they helped or hurt.
 - If you hit a wall on a step, write down what you tried and move on.
-

? Questions

Have fun! For all questions and clarifications, reach out without a thought 😊

Echo Team